CEE6501 — Lecture 8.2

Support Settlement

when $u_r \neq 0$

Learning Objectives

By the end of this lecture, you will be able to:

- Model support settlements within the Direct Stiffness Method
- Derive the modified partitioned equations when $u_r \neq 0$
- Interpret settlement using **imaginary restraints + superposition**
- Relate settlement to an equivalent joint load
- Correctly compute member forces and reactions

Part 1 — Conceptual Framework

Where Support Displacements Come From

In real structures, supports do not always remain perfectly fixed.

Small prescribed displacements may occur due to:

- Weak or compressible foundations
- Soil consolidation
- Differential settlement
- Fabrication tolerances or lack-of-fit

These are **imposed displacements**, not applied forces.

In DSM language:

$$u_r \neq 0$$

Core Question

How do we include prescribed support displacements within a force-based matrix framework?

DSM solves

$$Ku = F$$

Settlement does not directly enter as a force.

So we need a way to convert:

imposed displacements → equivalent forces

Key Idea

We use an imaginary fully restrained structure.

1. Restrain all joints.
2. Impose the prescribed settlements

$$\Delta_1,\ \Delta_2,\ \ldots$$

The fully fixed structure develops **fixed-joint forces**

$$F_1^S,\ F_2^S,\ F_2^S, \ldots$$

These are reactions required to enforce compatibility.

Imaginary Fully Fixed Structure

No description has been provided for this image

- Joints restrained
- Settlements imposed
- Fixed-joint forces develop

These forces represent the structural resistance to the imposed support movements.

Superposition Argument

We represent the actual system as:

$$
\begin{aligned}
&\text{Actual frame with settlement} \\
&= \\
&(1)\ \text{Fully fixed frame with settlement, develops } F^S \\
&+ \\
&(2)\ \text{Actual frame subjected to } -F^S
\end{aligned}
$$

What Does This Mean Physically?

- The fully fixed system develops forces F^S
- Applying their negatives to the real frame cancels out the artificial restraints
- What remains is the true compatibility-driven response

The negatives of the fixed-joint forces therefore act as:

Equivalent joint loads due to settlement

These loads produce the same joint displacements as the actual support settlements.

Part 2 — Global Equations

Global Equilibrium

The assembled system (including fixed-end forces) is

$$Ku = F - F^F$$

Partition the system:

$$\begin{bmatrix} K_{ff} & K_{fr} \\ K_{rf} & K_{rr} \end{bmatrix} \begin{Bmatrix} u_f \\ u_r \end{Bmatrix} = \begin{Bmatrix} F_f - F_f^F \\ F_r - F_r^F \end{Bmatrix}$$

Normally:

$$u_r = 0$$

With support settlement:

$$u_r \neq 0$$

Free DOFs Equation

From the first block row:

$$K_{ff}u_f + K_{fr}u_r = F_f - F_f^F$$

Rearranging:

$$K_{ff}u_f = F_f - F_f^F - K_{fr}u_r$$

Interpretation

What is $K_{fr}u_r$?

- u_r = imposed support displacements
- $K_{fr}u_r$ = forces induced in the free DOFs when those displacements are restrained

These are precisely the **fixed-joint forces** generated in the imaginary fully fixed structure.

Therefore,

$$F_f^S = K_{fr}u_r$$

is the mathematical form of the fixed-joint forces F^S, that you then subtract from the system to cancel out the artifical restraint.

Reactions

From the second block row:

$$F_r = K_{rf} u_f + K_{rr} u_r + F_r^F$$

The term, $K_{rr} u_r$, represents the contribution to the restrained DOFs due directly to the imposed settlement.

Again, this matches the fixed-frame interpretation: the fully restrained structure develops forces to enforce compatibility.

Big Picture Connection

Part 1 (Conceptual):

- Settlement → imaginary fully fixed structure
- Develop fixed-joint forces F^S
- Apply $-F^S$ to the real structure

Part 2 (Matrix Form):

- Settlement enters through u_r
- Fixed-joint forcesto be subtracted appear as

$$F^S = K_{fr} u_r$$

No new theory is needed. The superposition argument and the matrix formulation are the same mechanics as we used in definition FEFs in Lecture 6.

Part 3 — Support Settlement DSM Implementation

Code implementation steps

1. Assemble global K
2. Assemble applied load vector, F
3. Assemble fixed end force vector, F^F
4. Insert prescribed settlements into u_r
5. Partition matrices
6. Solve

$$u_f = K_{ff}^{-1}(F_f - F_f^F - K_{fr}u_r)$$

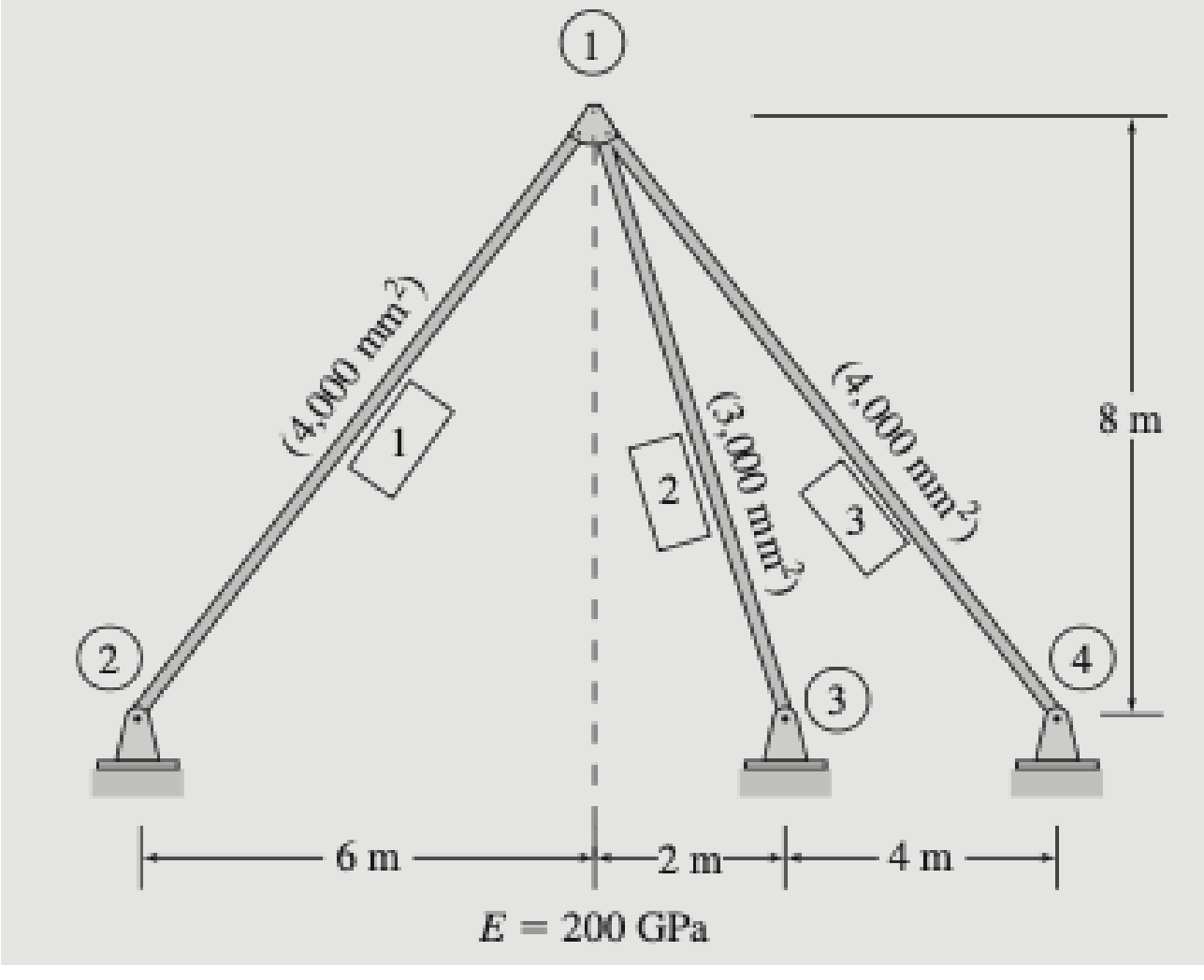
6. Compute reactions

$$F_r = K_{rf}u_f + K_{rr}u_r + F_r^F$$

7. Backward pass to recover element level forces

Part 4 — Example (7.2 in Kassimali)

Truss Structure



In [1]:

```
import sys
import os
import math
import numpy as np
import pandas as pd

# Get current notebook directory
current_dir = os.getcwd()

# Go up two levels, then into Code/L8
helpers_path = os.path.abspath(
    os.path.join(current_dir, "..", "..", "Code", "L8")
)

sys.path.append(helpers_path)
```

In [2]:
```
from helper_funcs_dsm import (
    assemble_global_stiffness_and_fef,
    partition_system,
    assemble_global_displacements,
    assemble_global_forces
)
from helper_funcs_output import (
    print_element_truss,
    print_dsm_results,
    print_matrix_scaled,
    plot_truss_deformation
)

# -------------------------
# Problem data (hard-coded)
# -------------------------
E = 200e6           # kN/m^2
A1 = 0.004 # m^2
L1 = math.sqrt(6**2 + 8**2)

A2 = 0.003 # m^2
L2 = math.sqrt(2**2 + 8**2)

A3 = 0.004 # m^2
L3 = math.sqrt(6**2 + 8**2)
```

In [3]:

```
# -------------------------
# Member 1, hard-coded
# -------------------------
theta = math.atan2(8,6)  # radians
c = np.cos(theta)
s = np.sin(theta)

factor = E * A1 / L1

k1 = factor * np.array([
    [ 1., 0., -1., 0.],
    [ 0., 0.,  0., 0.],
    [-1., 0.,  1., 0.],
    [ 0., 0.,  0., 0.]
], dtype=float)

# Local fixed-end vector 0 for Trusses
Qf1 = np.zeros(4, dtype=float)

T1 = np.array([
    [ c,  s, 0.,  0.],
    [-s,  c, 0.,  0.],
    [ 0.,  0., c,  s],
    [ 0.,  0.,-s,  c]
], dtype=float)

# Mapping
m1 = np.array([3,4,1,2])
```

In [4]:

```
# -------------------------
# Member 2, hard-coded
# -------------------------
theta = math.atan2(8,-2)
c = np.cos(theta)
s = np.sin(theta)

factor = E * A2 / L2

k2 = factor * np.array([
    [ 1., 0., -1., 0.],
    [ 0., 0.,  0., 0.],
    [-1., 0.,  1., 0.],
    [ 0., 0.,  0., 0.]
], dtype=float)

# Local fixed-end vector 0 for Trusses
Qf2 = np.zeros(4, dtype=float)

T2 = np.array([
    [ c,  s, 0.,  0.],
    [-s,  c, 0.,  0.],
    [ 0.,  0., c,  s],
    [ 0.,  0.,-s,  c]
], dtype=float)

m2 = np.array([5,6,1,2])
```

In [5]:
```
# -------------------------
# Member 3, hard-coded
# -------------------------
theta = math.atan2(8,-6)
c = np.cos(theta)
s = np.sin(theta)

factor = E * A3 / L3

k3 = factor * np.array([
    [ 1., 0., -1., 0.],
    [ 0., 0.,  0., 0.],
    [-1., 0.,  1., 0.],
    [ 0., 0.,  0., 0.]
], dtype=float)

# Local fixed-end vector 0 for Trusses
Qf3 = np.zeros(4, dtype=float)

T3 = np.array([
    [ c,  s, 0.,  0.],
    [-s,  c, 0.,  0.],
    [ 0.,  0., c,  s],
    [ 0.,  0.,-s,  c]
], dtype=float)

m3 = np.array([7,8,1,2])
```

In [6]:
```
k_list = [k1, k2, k3]
T_list = [T1, T2, T3]
Qf_list = [Qf1, Qf2, Qf3]
map_list = [m1, m2, m3]   # 1-based DOF maps

# Global system size (still using 1-based maps here)
ndof = int(np.max(np.concatenate(map_list)))

# Initialize Applied Load
F_global = np.zeros(ndof, dtype=float)

# Initialize Global Displacement
u_global = np.zeros(ndof, dtype=float)
```

In [7]:
```
## USER SPECIFIED

# Global Displacement (0-indexed)
u_global[8-1] = -0.01 #m

dof_restrained_1based = np.sort(
    np.array([3, 4, 5, 6, 7, 8], dtype=int)
)
```

In [8]:
```
K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
    )
```

In [9]:
```
print_matrix_scaled(K_global, scale=1000, decimals=2, col_width=5)
```

```
K = 1e+03 ×
1 | 61.88 -17.12 -28.80 -38.40 -4.28 17.12 -28.80 38.40
2 | -17.12 170.88 -38.40 -51.20 17.12 -68.48 38.40 -51.20
3 | -28.80 -38.40 28.80 38.40  0.00  0.00  0.00  0.00
4 | -38.40 -51.20 38.40 51.20  0.00  0.00  0.00  0.00
5 | -4.28 17.12  0.00  0.00  4.28 -17.12  0.00  0.00
6 | 17.12 -68.48  0.00  0.00 -17.12 68.48  0.00  0.00
7 | -28.80 38.40  0.00  0.00  0.00  0.00 28.80 -38.40
8 | 38.40 -51.20  0.00  0.00  0.00  0.00 -38.40 51.20
```

In [10]:

```
(
    K_ff,
    K_fr,
    K_rf,
    K_rr,
    f_f,
    f_r,
    u_r,
    f_fef_f,
    f_fef_r,
    free_dofs,
    restrained_dofs,
) = partition_system(
    K_global,
    F_global,
    u_global,
    F_fef_global,
    dof_restrained_1based
)
```

In [11]:
```
rhs = f_f - f_fef_f - K_fr @ u_r
u_f = np.linalg.solve(K_ff, rhs)

F_r = K_rf @ u_f + K_rr @ u_r + f_fef_r

u_global = assemble_global_displacements(
    u_f,
    u_r,
    free_dofs,
    restrained_dofs
    )

f_global_complete = assemble_global_forces(
    f_f,
    F_r,
    free_dofs,
    restrained_dofs
    )
```

In [12]:
```
print_dsm_results(
    u_global,
    f_global_complete,
    dof_restrained_1based,
    disp_in_mm=True
)
```

```
  DOF  Type Status  Disp (mm / rad)  Load (kN / kN·m)
    1   u_x   Free            5.530             0.000
    2   u_y   Free           -2.442             0.000
    3 theta  Fixed            0.000           -65.479
    4   u_x  Fixed            0.000           -87.306
    5   u_y  Fixed            0.000           -65.479
    6 theta  Fixed            0.000           261.917
    7   u_x  Fixed            0.000           130.958
    8   u_y  Fixed          -10.000          -174.611
```

In [13]:
```
# Element 1,2,3
print_element_truss(1, u_global, m1, T1, k1, Qf1, disp_in_mm=True, dec=2
print_element_truss(2, u_global, m2, T2, k2, Qf2, disp_in_mm=True, dec=2
print_element_truss(3, u_global, m3, T3, k3, Qf3, disp_in_mm=True, dec=2
```

```
E1 (Truss)
u_global [mm]: [0.00, 0.00, 5.53, -2.44]
u_local  [mm]: [0.00, 0.00, 1.36, -5.89]
q_local  [kN]: [-109.13, 0.00, 109.13, 0.00]
N (tension +) = 109.13 kN


E2 (Truss)
u_global [mm]: [0.00, 0.00, 5.53, -2.44]
u_local  [mm]: [0.00, 0.00, -3.71, -4.77]
q_local  [kN]: [269.98, 0.00, -269.98, 0.00]
N (tension +) = -269.98 kN


E3 (Truss)
u_global [mm]: [0.00, -10.00, 5.53, -2.44]
u_local  [mm]: [-8.00, 6.00, -5.27, -2.96]
q_local  [kN]: [-218.26, 0.00, 218.26, 0.00]
N (tension +) = 218.26 kN
```

In [14]:
```
plot_truss_deformation(nodes, elements, u_global, scale=100)
```

```
---------------------------------------------------------------
-----------
NameError                                 Traceback (most recent
call last)
Cell In[14], line 1
----> 1 plot_truss_deformation(nodes, elements, u_global, scale=
100)

NameError: name 'nodes' is not defined
```

Wrap-Up

Support settlement means:

$$u_r \neq 0$$

Core equation:

$$K_{ff}u_f = F_f - F_f^F - K_{fr}u_r$$

Settlement behaves like an equivalent joint load:

$$F_f^{(sett)} = -K_{fr}u_r$$

Superposition + compatibility drive the entire formulation.